Howto Recipes

Running AlphaFold 3 on Engaging

AlphaFold is an AI system developed by Google that is used for predicting protein structures. Here we provide a brief description of how to run this model on the Engaging computing cluster.

Getting Started

For simplicity, in this example, we will create a folder in our home directory to use as our working directory:

```
mkdir ~/af3
export WORKDIR=~/af3
```

Model weights

Each user needs to obtain the model weights individually due to licensing restrictions. These can be obtained by submitting a request to Google DeepMind. Usually, requests are granted within a few days. To make a request, follow the instructions on the AlphaFold 3 GitHub Repository.

When you get access, you will receive a link to download the parameters. After you download them, you can upload them to Engaging using `scp` on your local machine (you may receive a Duo push notification - see Transferring Files):

```
scp /path/to/source/af3.bin.zst $USER@orcd-login.mit.edu:~/af3
```

On Engaging, decompress the file and move to a `models` directory:

```
cd $WORKDIR
zstd -d af3.bin.zst
mkdir models
mv af3.bin models
rm af3.bin.zst
```

Container image

Google DeepMind provides instructions in their repository on running AlphaFold 3 with Docker. Docker is not compatible with most HPC environments, so we need to run a pre-built container using Apptainer. We have an image saved globally on Engaging, located at:

```
/orcd/software/community/001/container_images/alphafold3/20260318/alphafold3.sif
```

> **Note**
>
> The image that we have built uses a specific version of the AlphaFold 3 GitHub repository. As such, it does not change as their code base gets updated for bug fixes. If you would like us to update the image to reflect recent changes in the repository, let us know at orcd-help-engaging@mit.edu.

Running AlphaFold 3

The last thing you will need to run AlphaFold 3 is the AlphaFold dataset. Because it is quite large, we have saved it globally on Engaging for all users at `/orcd/datasets/001/alphafold3`.

Once you have everything you need, you will be ready to run AlphaFold 3. We will now go through a test case adapted from the AlphaFold 3 GitHub Repository. From the working directory, create an output directory and a test input file:

```
mkdir af_output
mkdir af_input
touch af_input/fold_input.json
```

Copy the following into `af_input/fold_input.json` (using `vim`, `emacs`, or `nano`):

fold_input.json

```
{
  "name": "2PV7",
  "sequences": [
    {
      "protein": {
        "id": ["A", "B"],
        "sequence":
"GMRESYANENQFGFKTINSDIHKIVIVGGYGKLGGLFARYLRASGYPISILDREDWAVAESILANADVVIVSVPINLTLETIERLKPYLT
      }
    }
  ],
  "modelSeeds": [1],
  "dialect": "alphafold3",
  "version": 1
}
```

You can either run this in an interactive session or in a batch job:

Interactive

Request an interactive session with a GPU:

```
salloc -N 1 -n 16 -p mit_normal_gpu -G 1
```

Run this script (`sh run_alphafold.sh`):

run_alphafold.sh

```
#!/bin/bash

module load apptainer

# Enter the path to the AF3 dataset and container image:
DATABASES_DIR=/orcd/datasets/001/alphafold3
IMAGE_PATH=/orcd/software/community/001/container_images/alphafold3/20260318/alphafold3.sif

# Enter the directory of the AF3 material:
WORKDIR=~/af3

apptainer exec \
    --bind $DATABASES_DIR:/root/public_databases \
    --nv \
    $IMAGE_PATH \
    python /app/alphafold/run_alphafold.py \
    --json_path=$WORKDIR/af_input/fold_input.json \
    --model_dir=$WORKDIR/models \
    --output_dir=$WORKDIR/af_output \
    --db_dir=/root/public_databases
```

Batch

Create your batch job script:

run_alphafold.sbatch

```
#!/bin/bash

#SBATCH -N 1
#SBATCH -n 16
#SBATCH -p mit_normal_gpu
#SBATCH -G 1

module load apptainer

# Enter the path to the AF3 dataset and container image:
DATABASES_DIR=/orcd/datasets/001/alphafold3
IMAGE_PATH=/orcd/software/community/001/container_images/alphafold3/20260318/alphafold3.si

# Enter the directory of the AF3 material:
WORKDIR=~/af3

apptainer exec \
    --bind $DATABASES_DIR:/root/public_databases \
    --nv \
    $IMAGE_PATH \
```

```
    python /app/alphafold/run_alphafold.py \
    --json_path=$WORKDIR/af_input/fold_input.json \
    --model_dir=$WORKDIR/models \
    --output_dir=$WORKDIR/af_output \
    --db_dir=/root/public_databases
```

Submit the batch job:

```
sbatch run_alphafold.sbatch
```

Output is saved to the `af_output` directory.

Note

Apptainer only binds your home directory by default. If your working directory is set to somewhere that is not in your home directory, then you will need to explicitly tell Apptainer to bind mount that directory. You can do this by adding `--bind $WORKDIR` to your `apptainer` command (after `exec` but before `$IMAGE_PATH`)

March 18, 2026